

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**МНОГООБЛАЧНО РАЗГРЪЩАНЕ НА
ИЗЧИСЛИТЕЛНО ИНТЕНЗИВНА УСЛУГА ОТ
ЕДНО ДЕКЛАРАТИВНО ОПИСАНИЕ НА
ИНФРАСТРУКТУРАТА И СРАВНИТЕЛНА
ОЦЕНКА НА ДВА ОБЛАЧНИ ДОСТАВЧИКА**

КУРСОВ ПРОЕКТ

по дисциплина „Облачни изчисления и технологии“

Разработил:

Алекс Цветанов, фак. № 121326006

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Доц. д-р Антония Ташева

София, 2027

СЪДЪРЖАНИЕ

УВОД	1
I. ПРЕДМЕТНА ОБЛАСТ И ОБОСНОВКА НА ИЗБОРА	2
1.1. Теоретична рамка	2
1.2. Направени избори	3
II. ПРОЕКТИРАНЕ НА СИСТЕМАТА	4
2.1. Единица работа и метрика за мащабиране	4
2.2. Политика за мащабиране	5
2.3. Договор на модулите, идентичност и уговорки	5
III. ПРОГРАМНА РЕАЛИЗАЦИЯ	6
IV. МЕТОДИКА И РЕЗУЛТАТИ	9
4.1. Обхват и условия на измерването	9
4.2. Капацитет спрямо брой реплики	9
4.3. Време за реакция при увеличаване	10
4.4. Поведение на контролера под натоварване	10
4.5. Разгръщане в Azure Container Apps	11
4.6. Какво разкри разгръщането	13
4.7. Разход и експлоатационни бележки	14
4.8. Какво не е измерено	14
ЗАКЛЮЧЕНИЕ	15
ЛИТЕРАТУРА	16
ПРИЛОЖЕНИЕ	17

УВОД

Публичните облачни платформи се представят като взаимозаменяеми: една и съща услуга може да бъде разгърната при кой да е доставчик, а изборът се свежда до цена. Проверката на това твърдение изисква едно и също натоварване да бъде разгърнато и измерено на две места при съпоставими условия, което рядко се прави.

Проектът изгражда *Stratus*: изчислително интензивна услуга в контейнер, обратен посредник, генератор на синтетично натоварване и контролер за мащабиране, всичките на C++20, заедно с описание на инфраструктурата, което насочва същата услуга към Amazon Web Services или Microsoft Azure чрез стойността на една променлива.

Целта е да се провери доколко едно декларативно описание на инфраструктурата е достатъчно за разгръщане на една и съща услуга при двама независими доставчика, и да се измери разликата между тях по закъснение, поведение при мащабиране и цена за единица работа.

Задачите са четири: кратка теоретична рамка и обосновка на направените технологични избори; проектиране, при което платформено зависимата част е сведена до преброими места; програмна реализация на услугата, наблюдението и мащабирането; методика за измерване и провеждане на измерванията.

Обхватът е ограничен до един тип изчислително натоварване и до двама доставчици. Извън обхвата остават хибридните и частните облаци, миграцията на състояние по време на работа и управляваните бази от данни.

Докъде е стигнала работата. Този абзац стои в увода, защото определя как да се четат всички числа по-нататък. Изградени и измерени са услугата, слой за наблюдение, контролерът за мащабиране и генераторът на натоварване, като част от измерванията са направени върху локален стек от контейнери на една машина. Описанието на инфраструктурата е **приложено при Azure** и услугата е измерена от външен клиент (Раздел 4.5). **При AWS не е разгръщано:** за тази сметка няма кредит, модулът е написан и форматиран, но никога не е прилаган, и число за закъснение, мащабиране или цена при AWS не съществува. Сравнение между двамата доставчици следователно липсва. Цената за единица работа също е неизмерима: Cost Management отказва абонамента Azure for Students с HTTP 422. Липсващите места са означени явно, вместо да бъдат запълнени с правдоподобни стойности.

I. ПРЕДМЕТНА ОБЛАСТ И ОБОСНОВКА НА ИЗБОРА

1.1. Теоретична рамка

Работното определение за *облачни изчисления* описва достъп при поискване до споделен резерв от ресурси и извежда пет характеристики: самообслужване, широк мрежов достъп, обединяване на ресурсите, бърза еластичност и измерена услуга [1]. Последните две са пряко относими тук: еластичността е това, което проверява синтетичното натоварване, а измерената услуга прави сравнението по цена възможно изобщо. Еластичността струва повече от средната утилизация, защото премахва нуждата от оразмеряване по върховото натоварване [2]. Трите модела на обслужване са IaaS (възли, мрежа и съхранение), PaaS (доставчикът поема средата за изпълнение) и SaaS (готово приложение). При управляваните услуги за контейнери границата между IaaS и PaaS е размита, и точно това прави сравнението нетривиално: еднакво наименована услуга може да лежи от различни страни на границата.

Технологичната основа на обединяването на ресурси е *виртуализацията*. Хардуерната виртуализация чрез хипервайзор изолира пълни виртуални машини със собствено ядро [3], докато контейнеризацията, тоест виртуализация на ниво операционна система, споделя едно ядро между изолирани пространства от имена и стартира многократно по-бързо за сметка на по-слаба изолация [4]. Изборът между двете определя и времето за реакция при мащабиране, и профила на риска.

Управлението на много еднакви възли води до *клъстерните технологии*: планировчик, който държи N реплики живи, каквато е и цялата нужда на този проект от оркестрация [5]. Предшественик са *GRID технологиите* за обединяване на ресурси между отделни организации [6]; разликата с облака е в собствеността и таксуването, не в техническата задача. Обектните хранилища жертват част от семантиката на файловата система срещу висока достъпност, като двама доставчици правят различен избор по оста достъпност спрямо съгласуваност [7, 8]. Сигурността се управлява по модела на споделената отговорност, чието следствие тук е отказът от дълготрайни ключове в полза на *управлявана идентичност*: възелът получава краткотраен маркер от самата платформа и никакъв секрет не се записва в хранилището с изходния текст.

1.2. Направени избори

Изборите са четири и всеки е оценен по три критерия, формулирани преди избора: наличие на съпоставим еквивалент при двамата доставчици, възможност описанието да остане едно, и пригодност на измерванията за сравнение. За изпълнение на работника са разгледани виртуални машини (пълен контрол, но стартиране от порядъка на минути), управляван Kubernetes и управлявани услуги за контейнери. За описание на инфраструктурата отпадат императивните скриптове и собствените средства на доставчиците, които работят само при своя доставчик; избран е Terraform, съответно OpenTofu, защото при него разликата се локализира в отделни модули. Три от изборите бяха преразгледани при реализацията, и Табл. 1 показва и първоначалния, и окончателния вариант (Табл. 1): премълчаването на първия би направило обосновката съчинена след факта.

Таблица 1. Направени избори, първоначални и след реализацията

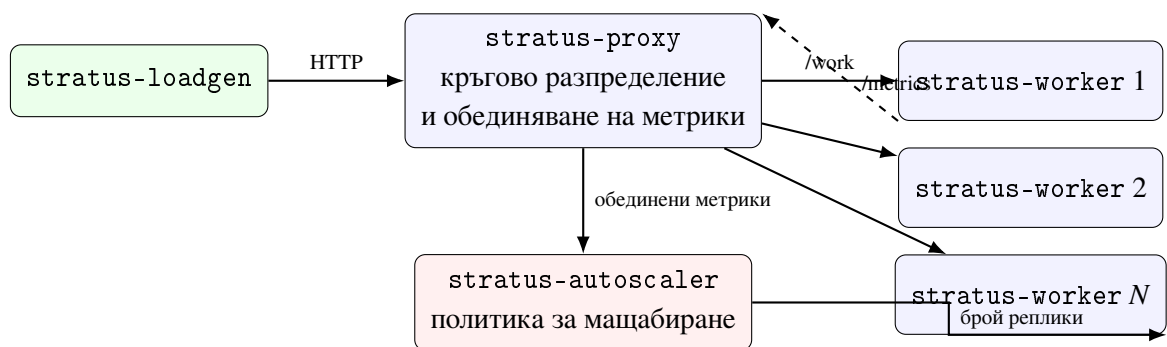
Решение	Първоначално	Окончателно	Причина и цена на промяната
Изпълнение на работника	Управляван Kubernetes, EKS и AKS	ECS с Fargate и Container Apps	Услугата не използва нищо от Kubernetes освен N живи реплики на едно име, а вторият слой на мащабиране замъглява измерваната величина
Описание на инфраструктурата	Terraform или OpenTofu	Без промяна	Няма
Генератор на натоварване	k6	Собствен, на C++	Нула задължителни външни зависимости. Цена: около 160 реда. Печалба: избор между затворен и отворен контур
Събиране на метрики	Prometheus и Grafana	Форматът на Prometheus, със собствено събиране	Изборът на <i>формат</i> запазва същественото: всеки скрейпър чете метриците без промяна в приложението. Цена: няма готово табло

Общото между трите промени е критерий, който не беше формулиран ясно в началото: изграждане с нула външни зависимости.

II. ПРОЕКТИРАНЕ НА СИСТЕМАТА

Системата е проектирана около едно ограничение: описанието на услугата трябва да е едно, а различията между доставчиците изнесени в преброими места. Разпръснат ли се различията из цялото описание, вече се сравняват две различни системи, а не две среди.

Реализацията се състои от пет изпълними файла върху една обща библиотека: `stratus-worker`, `stratus-proxy` (обратен посредник, който разпределя заявките кръгово и обединява метриките на набора реплики), `stratus-loadgen`, `stratus-autoscaler` и `stratus-cost`. Работникът не знае при кой доставчик работи: портът, размерът на пула от нишки и адресът на хранилището идват от променливи на средата. Потокът е следният (Фиг. 1): генераторът изпраща заявки към посредника, посредникът избира реплика по кръгов ред, всеки работник излага собствени броячи в текстов вид, посредникът ги сумира в един документ, а контролерът чете документа на равни интервали и решава дали да промени броя реплики.



Фигура 1. Структура на системата. Всичко на схемата е общо за двата доставчика

2.1. Единица работа и метрика за мащабиране

Сравнението по цена изисква знаменател, който не зависи от средата. За такъв е приета *една единица работа*: едно успешно изпълнение на `/work`. Изчислението е функцията за време на бягство на множеството на Манделброт върху решетка $n \times n$ с таван m итерации над фиксиран правоъгълник от комплексната равнина, сложност $O(n^2m)$. Прозорецът е фиксиран нарочно: преместването му би променило обема работа при същите параметри, без нищо във входа или изхода да покаже промяната. Стандартната единица е $n = 384$, $m = 500$, при която се изпълняват точно 18 690 064 итерации, проверени от модулен тест.

Работникът излага брояч `stratus_busy_seconds_total` и измерител

`stratus_worker_threads`, а контролерът изчислява натоварване $= \Delta \text{busy_seconds} / (\Delta t \cdot \sum \text{worker_threads})$, тоест както всяка система за наблюдение получава скорост от брояч. Работникът нарочно не изчислява отношението сам: то би носело неговия прозорец на осредняване, а не прозореца на контролера. Натоварването на процесора не се използва за сигнал: задачата насища ядро по построение, така че остава близо до единица и когато наборът се справя, и когато е задръстен.

2.2. Политика за мащабиране

Политиката е следене на цел с две допълнения. Желаният брой реплики е $r_{\text{жел}} = \lceil r_{\text{тек}} \cdot \text{натоварване} / \text{цел} \rceil$, ограничен между минимума и максимума. Първото допълнение е *зона на нечувствителност*: желание, различаващо се от текущия брой с по-малко от зададен дял, се приема за шум, иначе контролерът трепти без край. Второто са *несиметрични периоди на изчакване*: увеличаването е позволено след кратко изчакване, намаляването само след по-дълго, защото закъсняла реакция се плаща от всяка заявка в опашката, а закъсняло връщане на ресурс се плаща с една реплика.

2.3. Договор на модулите, идентичност и уговорки

Двата модула приемат един и същ обект, описващ услугата: име, образ, порт, изчислителен ресурс, памет, граници на броя реплики, целево натоварване, променливи на средата и етикети. Връщат едни и същи изходи: входна точка, обектно хранилище, платформа и идентичност. Заради този договор смяната на доставчика е смяна на стойността на една променлива. Пряко еднакви типове възли не съществуват, затова съпоставянето е записано преди измерванията: ресурсът се задава в единиците на AWS Fargate, където 1024 единици са едно ядро, а модулет за Azure дели на 1024.

Работникът никога не притежава дълготраен ключ: при AWS това е роля на задачата с права само върху една кофа, при Azure управлявана идентичност с роля за запис в един профил за съхранение, чиито ключове за споделен достъп са изключени.

Две уговорки. Първо, натоварването е чисто изчислително: модулите създават хранилището и подават името и идентичността чрез променливи на средата, но в работника няма клиент за обектно хранилище, защото обръщението би внесло мрежово чакане в единица работа, чиято цел е да бъде изцяло изчислителна. Второ, моделът на достъпа е *написан*, но не е *прилаган*: модулите само следват документираните схеми на ресурсите.

III. ПРОГРАМНА РЕАЛИЗАЦИЯ

Изходният текст е разделен по отговорност: `include/stratus/` съдържа логиката в заглавни файлове, `src/` реализациите и главните функции, `tests/` модулните тестове, `infra/` описанието на инфраструктурата. Указател към файловете е даден в приложението. Проектът се изгражда с нула задължителни външни зависимости: достатъчни са компилатор на C++20 и CMake. Двете места, където обичайно се появява зависимост, рамката за тестове и рамката за измерване, са заменени с малки собствени реализации.

Изчислително ядро и работник. Ядрото е свободно от заделяне на памет и връща контролна сума заедно с точния брой изпълнени вътрешни итерации. Контролната сума позволява проверка на отговора и не позволява на оптимизатора да премахне цикъл, чийто резултат никой не чете. Работникът излага `/healthz`, `/work?size=&iter=` и `/metrics`. Параметър извън допустимия обхват води до код 400, а не до подмяна с най-близката стойност: тихата подмяна би направила измерване с отхвърлен параметър да изглежда успешно.

HTTP сървърът използва пул с фиксиран размер, а не нишка на връзка. Изборът е съществен: при нишка на връзка едновременността е тази, която клиентът е избрал, процесорът се презаписва, а сигналът за натоварване престава да означава каквото и да било. При фиксиран пул капацитетът на една реплика е точно броят нишки в пула.

Преносимост. Единствената платформено зависима част е `src/net.cpp`, където се поемат разликите между Winsock и сокетите на BSD. Всичко над този файл е преносим C++20 без нито един условен блок за платформа.

Посредник и обединяване на метрики. Посредникът изпълнява две задачи, защото и на двете им трябва едно и също нещо: текущият списък с живи реплики. Той се получава чрез разрешаване на името на услугата, тъй като вграденият DNS връща по един адрес на работеща реплика. Няма регистър на услугите и няма презареждане на конфигурация. При обединяването етикетът с името на репликата се премахва и сериите с еднакъв ключ се сумират, което превръща N серии в една за набора.

Контролер. Самото решение е чиста функция и се проверява от пет модулни теста без работещ стек. Контролен цикъл, наблюдаем само чрез гледане на живо разгърнат клъстер, не може нито да бъде обсъждан, нито проверяван. Едно място заслужава коментар, защото първата му версия беше сгрешена: общата метрика е сума от броячите

на репликите, така че премахването на реплика кара сумата да *спадне*, а първата версия прочете отрицателната разлика като отрицателно натоварване и намали набора още повече. Правилното поведение е това на всяка система за наблюдение: отрицателна разлика в брояч е нулиране и интервалът се пропуска.

Генератор. При `--rate 0` контурът е затворен: всяка нишка изпраща следващата заявка веднага след връщането на предишната, тоест измерва се капацитет. При зададена скорост контурът е отворен: моментите на изпращане са предварително определени, така че забавена услуга натрупва време на чакане в отчетеното закъснение, вместо клиентът тихо да намали скоростта си. Персентилите се изчисляват върху пълната извадка, а не от кофите на хистограма, защото границата на кофа, отчетена като персентил, кара две различни системи да изглеждат еднакви.

Модел на разхода. Калкулаторът приема всяка цена като задължителен вход и отказва да работи без нея. В изходния текст няма нито една публикувана цена: цените се различават по регион, сметка и месец, така че цена, вписана в програма, е невярна още на следващия ден.

Тестове и контейнер. Тестовите са 26 и покриват детерминизма и цената на ядрото, разбора на заявки и параметри, представянето и разчитането на метриките, обединяването между реплики, петте случая на политиката, модела на разхода и статистиката за закъснението. Гетъри не се тестват. Всеки тест е отделен запис в CTest, така че при провал се съобщава кой тест е паднал. Образът се изгражда на два етапа: първият свързва изпълнимите файлове статично срещу `musl`, вторият е `scratch` и съдържа само тях; измереният размер е **8,09 MB**. Тестовите се изпълняват в етапа на изграждане: образ, който не минава тестовите си, не бива да съществува.

Описание на инфраструктурата. Кореновата конфигурация избира модул според стойността на променлива за доставчик и подава към него един и същ обект; изразите с `count` в `infra/main.tf` са единственото място, където доставчик се назовава извън модула си. Количествената мярка за преносимост, преброена в редове без празни и без коментари, е: обща част 82 реда, модул за AWS 297 реда, модул за Azure 178 реда. Съотношението не е в полза на твърдението, че едно описание покрива двамата доставчици: общата част е кратка, защото описва *услугата*, а мрежата, платформата, хранилището и идентичността се пишат два пъти. **Модулът за Azure е приложен** (девет ресурса в Italy North, после унищожени). **Модулът за AWS е написан и форматиран, но никога не е**

прилаган: за тази сметка няма кредит. В хранилището не се записват идентификатори на сметки или ключове.

IV. МЕТОДИКА И РЕЗУЛТАТИ

4.1. Обхват и условия на измерването

Този параграф определя какво изобщо може да се твърди. **Разгърнато е при един доставчик:** описанието на инфраструктурата е приложено в Azure и услугата е измерена от външен клиент (Раздел 4.5). **При AWS не е разгръщано:** за тази сметка няма кредит. Следователно число за закъснение, мащабиране или цена при AWS не съществува и не се съобщава, а сравнението между двамата доставчици липсва. Такова число не е и оценявано по аналогия: правдоподобната стойност е по-лоша от липсващата, защото изглежда като измерване. **Измерено е и локално:** три експеримента върху стек от контейнери на една машина. Те отговарят не на въпроса за разликата между доставчиците, а на друг: работи ли построената система, как се държи капацитетът ѝ при промяна на броя реплики, и реагира ли контролерът в разумно време.

Всички резултати са получени при Windows 11 Pro N (10.0.26200), 12 логически процесора, g++ 15.2.0 (MinGW-w64) в режим Release, CMake 4.3.2 с Ninja 1.13.2, Docker 29.4.3, образ върху Alpine Linux 3.20, изпълняван от scratch. Генераторът се изпълнява на *същата* машина, върху която работят контейнерите, и дели с тях същите 12 процесора. Това е основният източник на разсейване в локалните резултати.

Единицата работа е фиксирана за всички измервания: $n = 384$, $m = 500$. Всяка реплика има една нишка за обработка, така че капацитетът на набора е точно равен на броя реплики. Всяко измерване има 5 s загряване, чиито резултати се отхвърлят, и 20 s измерване, и се повтаря 3 пъти. Нищо не се осреднява преди да достигне до файл: съобщава се медианата, но и трите стойности са в results/.

4.2. Капацитет спрямо брой реплики

Броят реплики се задава на 1, 2, 4 и 6, при затворен контур с 2 едновременни заявки на реплика. Затвореният контур е избран нарочно: при него отчетената пропускателна способност е мярка за капацитет, а не мярка за търпението на клиента. Разсейването между повторенията е показано, защото е част от резултата (Табл. 2). Неуспешни заявки няма в дванадесетте повторения.

Таблица 2. Медиана от три повторения, с най-малката и най-голямата измерена стойност

Реплики	Пропуск. [зая./s]	Най-малка [зая./s]	Най-голяма [зая./s]	p50 [ms]	p90 [ms]	p99 [ms]
1	18,90	16,75	19,80	99,25	115,88	203,35
2	33,45	21,45	34,10	129,18	183,66	285,80
4	57,95	35,15	59,90	118,35	232,10	462,00
6	65,10	38,90	77,60	120,22	390,38	863,21

Коментар. Мащабирането е подлинейно и се изчерпва. От 1 до 4 реплики ускорението е 3,07 при четирикратен ресурс, тоест ефективност около 77%. От 4 до 6 реплики пропускателната способност нараства с 12%, а закъснението на 99-и персентил се удвоява, от 462 ms на 863 ms. Обяснението не е в услугата: при 6 реплики върху 12 логически процесора едновременно работят 6 изчислителни нишки, 64 нишки на посредника и 12 нишки на генератора, тоест системата е презаписана и се измерва планировчикът на операционната система. Разсейването расте заедно с броя реплики, затова локалните числа са характеристика на конкретната машина, а не на услугата. Медианното закъснение остава между 99 и 130 ms, докато високите персентили растат многократно: средната стойност при 6 реплики би скрила 863 ms опашка зад 120 ms медиана.

4.3. Време за реакция при увеличаване

Наборът се увеличава от 1 на 2 реплики и се измерва времето от подаването на командата до момента, в който посредникът започва да насочва заявки към новата реплика. При 5 повторения измерените стойности са 4,662, 5,234, 1,357, 2,001 и 1,840 s, тоест медиана 2,001 s. Времето съдържа стартиране на контейнера, регистрация на адреса във вградения DNS и откриването му от посредника, чийто период на опресняване е 1 секунда. Разликата от близо четири пъти при иначе еднакви условия показва, че политика с период на управление от порядъка на секунда работи с шум от същия порядък. Затова периодът в следващия експеримент е 3 секунди.

4.4. Поведение на контролера под натоварване

Стекът е вдигнат с една реплика, контролерът е стартиран с цел 0,70, граници от 1 до 6 реплики, период 3 s и изчакване 6 s при увеличаване и 15 s при намаляване.

Натоварването е в отворен контур, 40 заявки в секунда, 32 едновременни, 45 s измерване. Отчетът на генератора: 1868 успешни заявки, 0 неуспешни, 41,51 заявки/s, p50 = 51,84 ms, p90 = 1508,20 ms, p99 = 1639,47 ms. Високите персентили са опашката, натрупана в първите секунди, когато една реплика приема натоварване, което не може да поеме.

Таблица 3. Извадка от решенията на контролера. Пълната редица е в results/scaling-timeline.csv

<i>t</i> [s]	Реплики	Натоварване	Желани	Решение
6,9	1	0,000	1	задържане, на целта
10,3	1	1,101	2	увеличаване , над целта
15,6	2	0,848	2	задържане, изчакване преди увеличаване
20,2	2	0,822	3	увеличаване , над целта
25,6	3	0,996	3	задържане, изчакване преди увеличаване
33,3	3	0,667	3	задържане, на целта
57,9	3	0,646	3	задържане, на целта (десети пореден)
60,9	3	0,033	1	намаляване , под целта
65,9	1	нулиран брояч		пропуснат интервал

Коментар. Контурът се затваря (Табл. 3). Натоварването расте, контролерът увеличава набора на две стъпки, след което то се установява под целта 0,70 и решенията спират за десет интервала. След премахването на натоварването наборът се връща на една реплика. Установената стойност позволява независима проверка на цялата верига: при 3 реплики, натоварване 0,655 и 40,0 заявки в секунда времето за обслужване на заявка следва да е $0,655 \cdot 3 / 40,0 = 49,1$ ms, което съвпада с времето, което изчислителното ядро отчита само за себе си. Тоест метриката измерва извършената работа, а не чакането до нея.

Редовете при $t = 15,6$ и $t = 25,6$ показват периода на изчакване: без него контролерът щеше да добавя реплики, докато първите се стартират.

4.5. Разгръщане в Azure Container Apps

Средата на разгръщането е следната. Регион italnorth (Италия Север, Милано). Адрес stratus-dev.agreeablemeadow-0a00d2a7.italynorth.azurecontainerapps.io. Образ stratusacr4380.azurecr.io/stratus:v1. Контейнер с 1 vCPU и 2,00 GiB памет, при STRATUS_THREADS=1, тоест една изчислителна нишка на реплика. Мащабиране от 1

до 6 реплики с цел на натоварването 0,70. Клиентът е stratus-loadgen, изпълнен от София през публичния интернет, по обикновен HTTP на порт 80.

Тези числа не са сравними с локалните. Клиентът е в София, услугата е в Милано, а между тях стои публичният интернет. Закъснението съдържа пътя през глобалната мрежа и не бива да се поставя до таблица 2, където генераторът и контейнерите са на една машина.

Базовият пробег е с една връзка, 30 s и без загряване. Вторият е с 32 едновременни връзки, 10 s загряване и 240 s измерване, и отчита 1750,43 Miter/s извършена работа (Табл. 4).

Таблица 4. Azure Container Apps, два пробег от един и същи клиент. Суровите извадки от втория са в results/azure_scaling.csv

Показател	Базов пробег	Пробег с мащабиране
Успешни заявки	66	5698
Неуспешни заявки	0	47
Пропускателна способност [зая./s]	2,20	23,74
Средно закъснение [ms]	592,61	1282,74
p50 [ms]	120,24	1038,13
p90 [ms]	1129,83	2655,77
p95 [ms]	1133,61	3445,10
p99 [ms]	15157,82	15160,97
Най-голямо [ms]	15157,82	24152,75

Неуспешните заявки. 47 от 5745 заявки във втория пробег са неуспешни, тоест 0,82%. Разпределението не е равномерно: 29 от тях са между 30-ата и 60-ата секунда, докато цялото натоварване се поема от една реплика. Останалите 18 са разпръснати до края.

Опашката не идва от едновременността. p99 е около 15 s и в двата пробег, включително в базовия, където има само една връзка и 66 заявки. Стойност, която се появява еднакво при 1 и при 32 едновременни заявки, не се обяснява с натрупана опашка. Данните не позволяват избор между мрежата и студен старт, затова се съобщава само наблюдението.

Таблица 5. Брой реплики по време на пробег с мащабиране, секунди от началото на натоварването. Пълната редица е в `results/azure_measurement.txt`

<i>t</i> [s]	Репл.	<i>t</i> [s]	Репл.	<i>t</i> [s]	Репл.
3	1	108	2	230	3
60	1	155	2	249	3
99	1	221	2	268	3

Време за реакция. Това е най-полезното от пробег (Табл. 5). Първото увеличаване от 1 на 2 реплики е настъпило *между 99 и 108 секунди* след началото на натоварването. Второто, от 2 на 3 реплики, е настъпило *между 221 и 230 секунди*. Границата `max_replicas` от 6 реплики не е достигната в рамките на 240-секундния прозорец. Съобщава се интервал, а не момент: броят реплики се чете на около 9 до 10 секунди, така че действителната смяна е някъде между две последователни отчитания. Значение има порядъкът: реакцията е от порядъка на минута и половина, докато локалният контролер реагира за секунди (Табл. 3).

4.6. Какво разкри разгръщането

Дефект в самото описание на инфраструктурата. Във файла `infra/versions.tf` стоеше коментар, че двата доставчика могат да се конфигурират безусловно, защото „нито един не се удостоверява, преди действително да се планира ресурс, така че конфигурирането на неизползвания не струва нищо“. Това е невярно. Доставчикът за AWS търси данни за достъп при конфигурирането, а не при планирането на ресурс. Затова `terraform plan` за целта Azure изчисляваше пълния план от девет ресурса за Azure и после се проваляше поради липсващи данни за достъп до AWS, при това след изчакване на точката за метаданни на EC2 на адрес `169.254.169.254`. Грешката е стояла незабелязана, защото `terraform` никога не е бил изпълняван. Поправена е с трите документиран аргумента `skip_*` и защитни фиктивни ключове, всички обусловени от `target_provider`, така че действителната верига за удостоверяване остава непокътната, когато целта е AWS.

Наемателят забранява удостоверяване със споделен ключ за хранилищата. Прилагането се проваляше с `KeyBasedAuthenticationNotPermitted`, след като сметката за хранилище вече е създадена. Модулът беше правилен: в него `shared_access_key_enabled` е `false`. Вината е у доставчика, който по подразбиране

проверява равнището за данни със споделен ключ. Поправено с `storage_use_azuread = true` на доставчика `azurerms`.

Azure for Students ограничава регионите. Правило с име `sys.regionrestriction` разрешава само `francecentral`, `switzerlandnorth`, `spaincentral`, `italynorth` и `denmarkeast`. Първият опит, в `polandcentral`, беше отказан. Избран е `italynorth`, най-близкият разрешен регион до София.

4.7. Разход и експлоатационни бележки

Разходът се вижда, но не се разпределя. Наличният кредит спадна от 100 на 88 USD за около един час, през който разгръщането беше вдигнато, докато полето „разходи за август“ в портала показваше 0,00. Тоест 12 USD са изразходвани и се виждат в остатъка от кредита, но не са отнесени към услуга. При това темпо кредитът стига за часове, а не за 8760-те часа в годината, за които изглежда предвиден. Затова всичко е унищожено веднага след пробегата.

Ингресът беше превключен ръчно. Изпълнена е командата `az containerapp ingress update --allow-insecure`, извън `terraform` и нарочно. Генераторът на натоварване в този проект говори обикновен HTTP, а ингресът иначе отговаря на порт 80 с пренасочване 301. Промяната е извън описанието на инфраструктурата, за да запази то поведението си само по HTTPS.

4.8. Какво не е измерено

Няма сравнение между двама доставчици. Това е най-тежкото ограничение на документа. При AWS не е разгръщано, защото за тази сметка няма кредит. Замисълът, едно описание при двама доставчици и измерена разлика между тях, остава проверен наполовина.

Цената за единица работа остава неизмерена, и то по причина, която не се решава с още време. Програмният интерфейс Cost Management отказва тази абонаментна сметка направо, с код HTTP 422: той поддържа само типовете Enterprise Agreement, Web direct и Microsoft Customer Agreement, а Azure for Students не е нито един от трите. Остава само порталът, който показва обобщена стойност на кредита и изоставя спрямо потреблението. Тоест числото не е просто незаписано, а е недостъпно през интерфейса при този тип абонамент.

ЗАКЛЮЧЕНИЕ

Какво е проверено и какво не. Системата е измерена локално: капацитет спрямо брой реплики, време за реакция при увеличаване и поведение на контролера под натоварване, всяко с повторения и със записани сурови данни. Описанието на инфраструктурата е приложено **веднъж**, при Azure, и разгърнатата услуга е измерена от външен клиент (Раздел 4.5). При AWS не е разгръщано, защото няма кредит за тази сметка. Следователно измерване на разликата *между* двете среди в този документ няма, и такова не е съчинено.

Предимства. Метриките се събират от самото приложение, с един и същ код при всяко разгръщане, така че разликата в измереното не се дължи на разлика в измервателния уред. Политиката за мащабиране е чиста функция и се проверява без работещ клъстер, което позволи трите ѝ нетривиални поведения да бъдат обсъдени по измерени редове. Проектът се изгражда без нито една задължителна външна зависимост, полученият образ е 8,09 MB, а в хранилището не се съхранява нито един секрет.

Ограничения. Твърдението „едно описание за двама доставчици“ се оказва по-слабо, отколкото звучи: 82 реда обща част срещу 297 и 178 реда платформено зависими модули, тоест дели се описанието на *услугата*, а не на средата. Това е резултат от преброяване на редове, не от измерване при работещо разгръщане. Локалните измервания са направени на машина, която едновременно изпълнява генератора, посредника и всички реплики, тоест при 6 реплики системата е презаписана.

Едно очакване, което не се потвърди. Първоначалният избор беше управляван Kubernetes, с обосновката, че описанието на работното натоварване съвпада буквално при двамата доставчици. При реализацията се оказва, че услугата не използва нищо от Kubernetes освен способността му да държи N реплики достъпни на едно име, а добавя втори слой на мащабиране, който замъглява измерваната величина. Първоначалната обосновка е запазена в Раздел I заедно с преразглеждането, вместо да бъде премълчана.

Насоки за по-нататъшна работа. Прилагане и на модула за AWS, с което сравнението между двамата доставчици най-после ще има числа, след това втори тип натоварване, при което гаранциите на обектното хранилище влизат в сила. Цената за единица работа изисква абонамент, чийто тип се приема от програмния интерфейс Cost Management.

ЛИТЕРАТУРА

- [1] P. Mell and T. Grance, “The NIST definition of cloud computing,” Tech. Rep. Special Publication 800-145, National Institute of Standards and Technology, Gaithersburg, MD, USA, September 2011.
- [2] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, and M. Zaharia, “A view of cloud computing,” *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [3] P. Barham, B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, R. Neugebauer, I. Pratt, and A. Warfield, “Xen and the art of virtualization,” in *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP '03)*, pp. 164–177, 2003.
- [4] D. Merkel, “Docker: Lightweight Linux containers for consistent development and deployment,” *Linux Journal*, vol. 2014, no. 239, 2014.
- [5] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, “Large-scale cluster management at Google with Borg,” in *Proceedings of the 10th European Conference on Computer Systems (EuroSys '15)*, 2015.
- [6] I. Foster, C. Kesselman, and S. Tuecke, “The anatomy of the grid: Enabling scalable virtual organizations,” *International Journal of High Performance Computing Applications*, vol. 15, no. 3, pp. 200–222, 2001.
- [7] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels, “Dynamo: Amazon’s highly available key-value store,” in *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP '07)*, pp. 205–220, 2007.
- [8] B. Calder, J. Wang, A. Ogus, N. Nilakantan, A. Skjolsvold, S. McKelvie, Y. Xu, S. Srivastav, J. Wu, H. Simitci, *et al.*, “Windows Azure storage: A highly available cloud storage service with strong consistency,” in *Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP '11)*, pp. 143–157, 2011.

ПРИЛОЖЕНИЕ

Пълният изходен текст е публичен и се намира на адрес <https://github.com/Alex-Tsvetanov/stratus>. Тук е даден указател към него, а не копие: хранилището се изгражда с една команда, а разпечатан код не се чете.

Таблица 6. Указател към изходния текст. Броят редове е преброен с `wc -l`, с празните редове и коментарите

Файл	Съдържание	Редове
include/stratus/mandelbrot.hpp	Изчислителното ядро, единицата работа	66
include/stratus/scaling.hpp	Политиката за мащабиране, чиста функция	107
include/stratus/metrics.hpp	Броячи, хистограма, текстово представяне	187
include/stratus/promparse.hpp	Разчитане и обединяване на метрики	149
include/stratus/cost.hpp	Модел на разхода, без вградена цена	81
include/stratus/http.hpp	Разбор на заявки и отговори	192
include/stratus/net.hpp, src/net.cpp	Сокети, единственото място с Winsock и BSD	328
include/stratus/httpd.hpp, src/httpd.cpp	Сървър с пул от нишки с фиксиран размер	130
include/stratus/args.hpp, stats.hpp	Разбор на аргументи, персентили	105
src/worker_main.cpp	Работникът, три крайни точки	179
src/proxy_main.cpp	Посредник и обединяване на метрики	203
src/loadgen_main.cpp	Генератор, затворен и отворен контур	166
src/autoscaler_main.cpp	Контролерът за мащабиране	212
src/cost_main.cpp	Калкулаторът за цена	74
tests/test_framework.hpp	Рамката за тестове, вместо външна зависимост	111
tests/test_main.cpp	26 теста	443
Dockerfile, docker-compose.yml	Два етапа, scratch, локален стек	122
infra/ (обща част)	Обектът на услугата, избор на модул, изходи	126
infra/modules/aws/main.tf	ECS Fargate, ALB, S3, IAM. Не е прилаган	386
infra/modules/azure/main.tf	Container Apps, Storage. Не е прилаган	237

Суровите резултати са в директорията `results/`: пропускателна способност и закъснение за всяко повторение, по един ред за всяка заявка, времената за реакция и решенията на контролера. Обработката им в Раздел IV е медиана по три повторения.